

Temporal Data Models

Temporal Data Models

- Temporal Data Models: extension of relational model by adding temporal attributes to each relation
- Temporal Query Languages: TQL, SQL3



Structure of Temporal Data Models

- Entities – real-world objects (Employee, Project)
- Attributes – descriptive fields (salary, address)
- Temporal Attributes – valid time, transaction time
- Time Intervals – represented as $\langle \text{start_time}, \text{end_time} \rangle$

Features of Temporal Data Models

- Support for time-varying data
- Retain history and query past/future states
- Bi-temporal consistency
- Temporal integrity constraints
- Time-based queries (AS OF, BETWEEN)

Time Specification in SQL-92

- **date**: four digits for the year (1--9999), two digits for the month (1--12), and two digits for the date (1--31).
- **time**: two digits for the hour, two digits for the minute, and two digits for the second.
- **timestamp**: the fields of **date** and **time**, with six fractional digits for the seconds field.
- **interval**: refers to a period of time (e.g., 2 days and 5 hours), without specifying a particular time when this period starts; could more accurately be termed a *span*.

Types of Temporal Models

- Snapshot Model – Stores only current data
- Historical Model – Keeps past and current data
- Rollback Model – Allows rollback to any past state
- Bi-Temporal Model – Maintains both valid and transaction time

The time stamp model

- All the tuples in a relation have an additional temporal attribute
- Example: **Booking (meeting, room, time)**
 - A tuple (m,r,t) denotes the fact that:
meeting m is in room r at time t

Booking		
<i>Meeting</i>	<i>Room</i>	<i>Time</i>
DB Group	DC1331	06-Jan-04.10:00
DB Group	DC1331	06-Jan-04.10:01
DB Group	DC1331	06-Jan-04.10:02
...	...	...
DB Group	DC1331	16-Jan-04.11:59
Intro to Databases	MC4042	06-Jan-04.10:00
...	...	...
Intro to Databases	MC4042	06-Jan-04.11:19
Intro to Databases	MC4042	08-Jan-04.10:00
...	...	...
Intro to Databases	MC4042	08-Jan-04.11:19

The snapshot model

- Different view from the time stamp model (of the same data)

<i>Time</i>	<i>booking</i>
06-Jan-04.10:00	{ (DB Group,DC1331), (Intro to Databases,MC4042) }
06-Jan-04.10:01	{ (DB Group,DC1331), (Intro to Databases,MC4042) }
...	
06-Jan-04.11:19	{ (DB Group,DC1331), (Intro to Databases,MC4042) }
06-Jan-04.11:20	{ (DB Group,DC1331) }
...	
06-Jan-04.11:59	{ (DB Group,DC1331) }
06-Jan-04.12:00	{ }
...	
06-Jan-04.12:00	{ }
08-Jan-04.10:00	{ (Intro to Databases,MC4042) }
...	
08-Jan-04.11:19	{ (Intro to Databases,MC4042) }

Temporal Databases

Temporal DBs – Motivation

- Conventional databases represent the state of an enterprise at a single moment of time
- Many applications need information about the past
 - Financial (payroll)
 - Medical (patient history)
 - Government
- Temporal DBs: a system that manages time varying data

Comparison

Conventional DBs:

- Evolve through **transactions** from one state to the next
- Changes are viewed as modifications to the state
- No information about the past
- Snapshot of the enterprise

Temporal DBs:

- Maintain **historical** information
- Changes are viewed as additions to the information stored in the database
- Incorporate notion of time in the system
- Efficient access to **past** states

Taxonomy of Temporal DBs

- **Single-dimensional:** temporal relations were allowed only a single temporal attribute
- **Multiple dimensional:** with each tuple in a relation there can be more than one temporal attribute
 - Example: two kinds of time are stored: the **valid time** (when a particular tuple is true) and the **transaction time** (when the particular tuple was inserted/deleted in the database)

Characteristics of Temporal DB

- **Valid time** is the time that a fact becomes effective in reality.
- The **transaction time** for a fact is the time interval during which the fact is current within the database system.
- In a **temporal relation**, each tuple has an associated time when it is true; the time may be either valid time or transaction time.
- A **bi-temporal relation** stores both valid and transaction time.

Characteristics of Temporal DB(Cont.)

- Example of a temporal relation:

<i>account-number</i>	<i>branch-name</i>	<i>balance</i>	<i>from</i>	<i>to</i>
A-101	Downtown	500	1999/1/1 9:00	1999/1/24 11:30
A-101	Downtown	100	1999/1/24 11:30	*
A-215	Mianus	700	2000/6/2 15:30	2000/8/8 10:00
A-215	Mianus	900	2000/8/8 10:00	2000/9/5 8:00
A-215	Mianus	700	2000/9/5 8:00	*
A-217	Brighton	750	1999/7/5 11:00	2000/5/1 16:00

- Temporal query languages have been proposed to simplify modeling of time as well as time related queries.

Example

- Sales example: data about sales are stored at the end of the day
- Transaction time is different than valid time
- Valid time can refer to the future also!
 - Credit card: 03/13-04/16
- Transaction Time evolves discretely, usually is associated with the transaction number:
$$T1 \rightarrow T2 \rightarrow T3 \rightarrow T4 \dots$$
- A record R is extended with an interval [t.start, t.end). When we insert an object at t1 the temporal attributes are updated $\rightarrow [t1, \text{now})$

Intervals in Temporal DB

- An *interval* [s,e] is a set of times from time s to time e.
 - *Time Quanta*: smallest unit of time system can represent.
 - *Timepoints/point*: time unit considered indivisible for our purpose.
- An interval is treated as a single type, not as pair of separate values.
- Interval can be open/closed w.r.t. start point/end point.
 - eg. [d04,d10],[d04,d11),(d03,d10],[d03,d11)all represent the sequence of days from day4 to day10 inclusive.

Intervals

Notation	Meaning	Example (days)
$[s, e]$	includes start and end	$\{d04, \dots, d10\}$
(s, e)	excludes both start and end	$\{d05, \dots, d09\}$
$[s, e)$	includes start, excludes end	$\{d04, \dots, d09\}$
$(s, e]$	excludes start, includes end	$\{d05, \dots, d10\}$

Operators on Intervals

- Temporal predicate operators:

$i1 = [s1, e1]; i2 = [s2, e2]$

– **i1 BEFORE i2**

$(e1 < s2)$



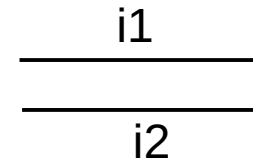
– **i1 MEETS i2**

$(s2 = e1)$



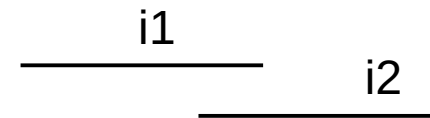
– **i1 EQUALS i2**

$(s1 = s2 \text{ AND } e1 = e2)$



– **i1 OVERLAPS i2**

$(s2 < s1 < e2 \text{ OR } s1 < s2 < e1)$



Operators on Intervals

- **i1 DURING i2**

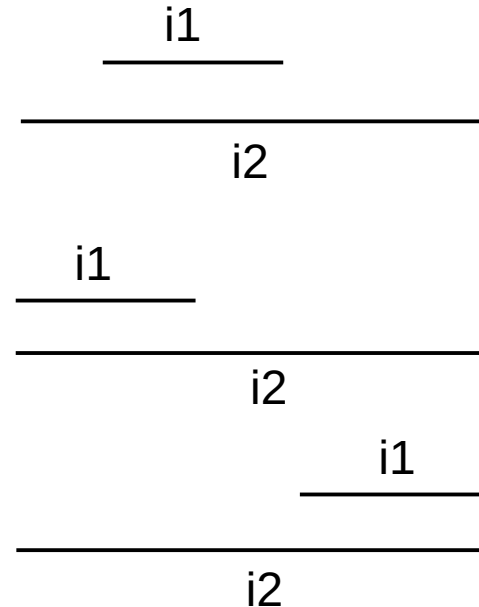
$(s_2 < s_1 \text{ AND } e_2 > e_1)$

- **i1 STARTS i2**

$(s_1 = s_2 \text{ AND } e_1 < e_2)$

- **i1 FINISHES i2**

$(e_1 = e_2 \text{ AND } s_1 > s_2)$



- Additional operators:

- **i1 MERGES i2:** $(i1 \text{ MEETS } i2 \text{ OR } i1 \text{ OVERLAPS } i2)$

- **i1 CONTAINS i2:** $(i2 \text{ DURING } i1)$

Scalar and Relational Operators

- **DURATION(i)** - returns the number of time points in i
 - eg. DURATION ([d03,d07]) returns 5
- **i1 UNION i2**
 - returns [MIN(s1,s2),MAX(e1,e2)]
if (i1 MERGES i2)
otherwise undefined
- **i1 INTERSECT i2**
 - returns [MAX(s1,s2),MIN(e1,e2)]
if (i1 OVERLAPS i2)
otherwise undefined

Aggregate Operators

- EXPAND(X):

Where X is a set. The output is also a set.

Used to generate time quantum intervals.

– The *expanded form* of X is the set of all intervals of the form [p,p] where p is a time point in some interval in X.

- e.g.:

– $X1 = \{ [d01,d01],[d03,d05],[d04,d06] \}$

– $X2 = \{ [d01,dp1],[d03,d04],[d05,d05],[d05,d06] \}$

– $X3 = \{ [d01,d01],[d03,d03],[d04,d04],[d05,d05],[d06,d06] \}$

– Then $EXPAND(X1) = EXPAND(X2) = X3$

Aggregate Operators

- **COLLAPSE(X):**

The *collapsed form* of X is the set Y of intervals of the same type such that

- (a) X & Y have the same unfolded form.
- (b) no two distinct members i1 and i2 of Y are such that (i1 MERGES i2) is true.

- e.g.:

- $X1 = \{ [d01,d01],[d03,d05],[d04,d06] \}$
- $X2 = \{ [d01,d01],[d03,d04],[d05,d05],[d05,d06] \}$
- $X3 = \{ [d01,d01],[d03,d06] \}$

- Then $\text{COLLAPSE}(X1) = \text{COLLAPSE}(X2) = X3$

Extensions of SQL based on interval based language

- TSQL2 or SQL/Temporal

```
select r1.Meeting
from   Booking r1, Booking r2
where  r1.Meeting = r2.Meeting
       and r1.time before r2.time
```

- Time attributes range over intervals and the before relationship denotes the before relationship between two intervals

Temporal Relational Algebra

Temporal Relational Algebra (TRA)

- Extension of classical relational algebra to handle time-dependent data.
- Each tuple is associated with temporal attributes such as valid time and transaction time.
- Operations consider both data and their time intervals.
- Basis for temporal query languages (e.g., TSQL2).

Temporal Relations

- Temporal relations include timestamps with each tuple.

- Example:

Employee(EID, Name, Department, Valid_From, Valid_To)

- Allows querying data valid during a specific time interval.

Temporal Query Languages

- **Temporal selection:** involves time attributes
- **Temporal projection:** the tuples in the projection inherit their time-intervals from the tuples in the original relation.
- **Temporal join:** the time-interval of a tuple in the result is the intersection of the time-intervals of the tuples from which it is derived. If intersection is empty, tuple is discarded from join.

Temporal Selection (σ_T)

- Select tuples valid during a specific time interval.
- Syntax: $\sigma_T[\text{condition, time-interval}](\text{Relation})$
- Example:

$\sigma_T[\text{Department}='HR', [2019,2021]](\text{Employee})$

→ Selects employees in HR valid between 2019 and 2021.

Temporal Projection (π_T)

- Projects attributes and their corresponding time intervals.
- Syntax: $\pi_T[\text{attributes}](\text{Relation})$
- Example:

$\pi_T[\text{EID}, \text{Department}](\text{Employee})$

→ Retrieves employee IDs and departments with valid time intervals.

Temporal Join ($\bowtie_T$)

- Combines tuples from two temporal relations where time intervals overlap.
- Syntax: $R \bowtie_T S$
- Example:

Employee $\bowtie_T$ Department

→ Returns tuples valid during the overlapping periods of both relations.

Other Temporal Operations

- Temporal Union ($\cup_T$): Combines tuples valid at any overlapping time.
- Temporal Intersection ($\cap_T$): Keeps tuples valid during common time periods.
- Temporal Difference ($-_T$): Removes tuples valid during certain periods.
- Temporal Aggregation: Summarizes data over time intervals.

Example Query

Query 1: Find all employees who worked in 'Finance' during 2020.

→ $\sigma_{T[\text{Department}=\text{'Finance'}, [2020,2020]]}(\text{Employee})$

Query 2: Find employees who worked in both HR and IT at overlapping periods.

→ $(\sigma_{T[\text{Department}=\text{'HR'}]}(\text{Employee})) \bowtie_T (\sigma_{T[\text{Department}=\text{'IT'}]}(\text{Employee}))$

Spatial Database

Spatial Database

- Spatial databases support information about objects in multidimensional space
 - Examples: geographic information systems, weather information databases
- Spatial relationships among the objects are important
- Optimized to query data such as points, lines, and polygons
 - Spatial queries

Spatial Database (Cont.)

Spatial data types

- n **Map data**

- n Geographic or spatial features of objects in a map

- n **Attribute data**

- n Descriptive data associated with map features

- n **Image data**

- n Satellite images

Spatial Database (Cont.)

Spatial operator categories

- n **Topological operators**

- n Properties do not change when topological transformations applied. Examples: Inside / Contains, Adjacent, Touches

- n **Projective operators**

- n Express **concavity/convexity** of objects. Examples: Concavity/Convexity Test

- n **Metric operators**

- n Specifically describe object's geometry. Examples: Distance, Area

- n **Dynamic spatial operators**

- n Used when spatial objects change over time, including movement, growth, or deformation. Examples: Create, destroy, and update

Spatial Database (Cont.)

n Spatial queries

n Range queries

n Retrieve all spatial objects that lie **within a given region or boundary** (rectangle, circle, polygon, etc.).

Example: find all hospitals with the Metropolitan Atlanta city area

- A query region is defined (e.g., “Atlanta city area”).
- Each object is checked to see if its geometry lies inside or overlaps with the region.
- Useful Operators: Inside(region, object), and Intersects(region, object)

Spatial Database (Cont.)

n Spatial queries (Cont.)

n Nearest neighbor queries

n Identify the **closest spatial object** to a given point or location.

Example: find police car nearest location of a crime

- Computes distance using Euclidean or network distance.
- Spatial indexes (e.g., R-tree) speed up the search.
- Useful Operators: Nearest(object, set)

Spatial Database (Cont.)

n Spatial queries (Cont.)

n Spatial joins or overlays

n Combine two spatial datasets based on **spatial relationships** like “within”, “intersects”, “overlaps”, etc.

Example: find all homes within two miles of a lake

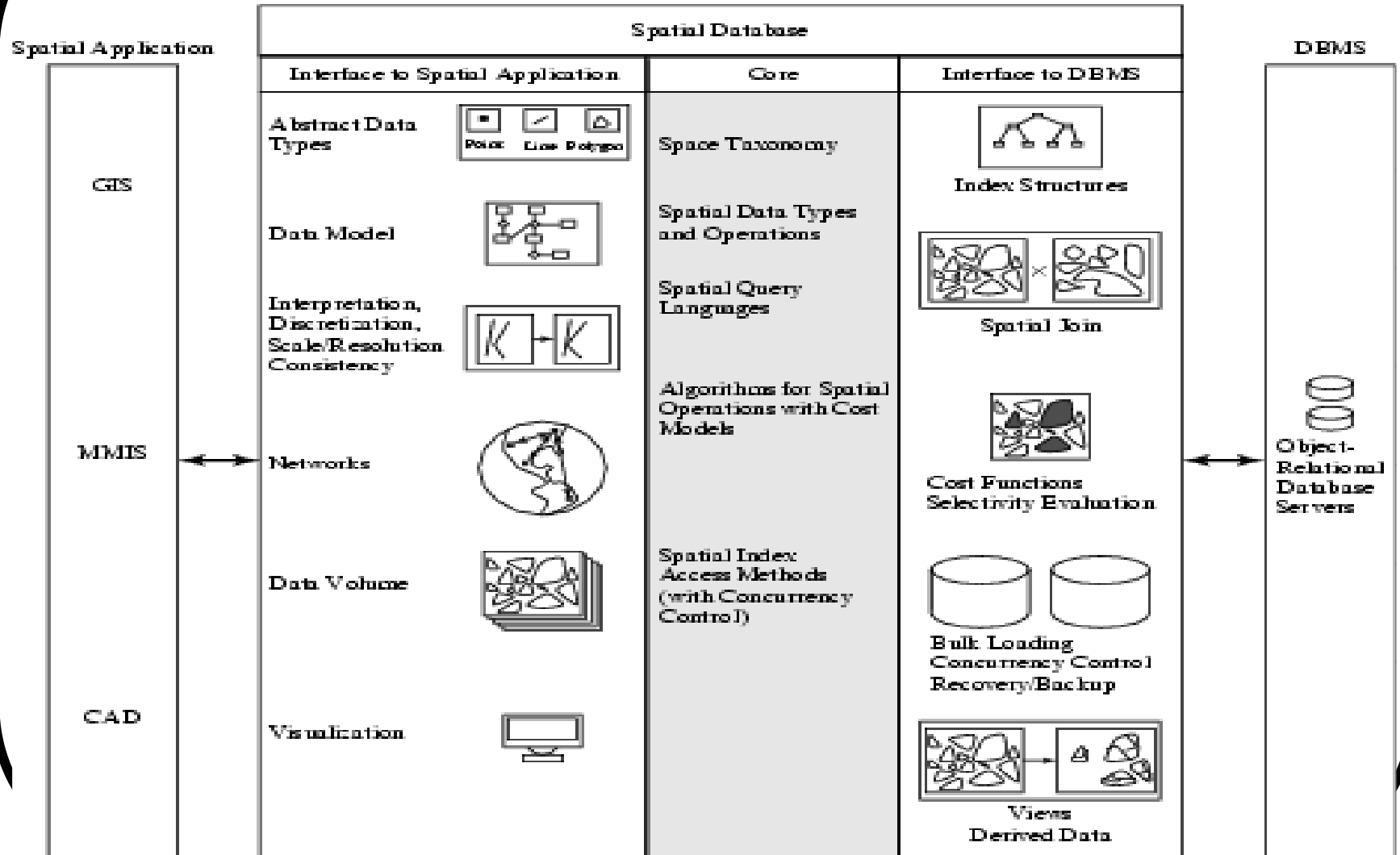
- For each object in Dataset A, check its spatial relation with objects in Dataset B.
- Often uses spatial indexing for performance.
- Useful Operators: Within(A, B), Intersects(A, B), Touches(A, B), and Overlaps(A, B)

Spatial Database (Cont.)

- Consider a spatial dataset with:
 - County boundary (dashed white line)
 - Census block - name, area, population, boundary (dark line)
 - Water bodies (dark polygons)
 - Satellite Imagery (gray scale pixels)
- Storage in a SDBMS table:
create table **census_blocks** (
 name string,
 area float,
 population number,
 boundary polygon);



Spatial Database (Cont.)



Spatial Database (Cont.)

- Spatial Databases integrate **geometric (spatial)** and **attribute (non-spatial)** data.
- Applications: **GIS** (Geographic Information Systems), **CAD** (Computer-Aided Design), **MMIS** (Municipal Management Information Systems) use these databases for spatial analysis.
- Architecture includes three main interfaces:
 - Interface to Spatial Applications
 - Core Spatial Database Components
 - Interface to DBMS

Spatial Database (Cont.)

Interface to Spatial Applications

Key Components:

Abstract Data Types: Point, Line, Polygon

Data Model: Defines spatial relationships (topology, geometry)

Interpretation & Scale Consistency: Ensures accurate representation across resolutions

Networks: Handles spatial connectivity (roads, rivers, etc.)

Data Volume: Manages large-scale geospatial data

Visualization: Converts spatial data into graphical form

Spatial Database (Cont.)

Core of Spatial Database

Includes:

Space Taxonomy: Defines spatial entities and relationships

Spatial Data Types & Operations: Provides geometric operations (e.g., area, distance)

Spatial Query Languages: SQL extensions for spatial queries

Algorithms with Cost Models: Optimizes spatial query performance

Spatial Indexing: Improves retrieval (e.g., R-trees, Quad-trees)

Spatial Database (Cont.)

Interface to DBMS

Functions:

Index Structures: Spatial indexing for efficient access

Spatial Join: Combines spatially related objects

Cost Functions & Selectivity Evaluation: Optimizes query planning

Bulk Loading & Concurrency Control: Manages parallel access and recovery

Views & Derived Data: Supports materialized spatial views

Spatial Database (Cont.)

Integration with DBMS

Key Point:

Spatial DB connects with **Object-Relational Database Servers** for:

- Persistent storage

- Transaction management

- Backup & recovery